Cryptographic Hash Functions

Jean-Michel Dricot – ULB Cybersecurity Research Center

In This Lecture

- Hashing functions
- Challenge-response authentication
- Message integrity
- Cooldown mechanisms
- Lightweight key derivations
- Secure MAC constructions

Definitions

Hash function

A hash function $H()$ accepts a variable-length block of data M as input and produces a **fixed-size** hash value $h = H(M)$

Cryptographic hash function

An algorithm for which it is computationally infeasible to find either: (1) a data object that maps to a pre-specified hash result (the **one-way property**) or (2) two data objects that map to the same hash result (the **collision-free property**)

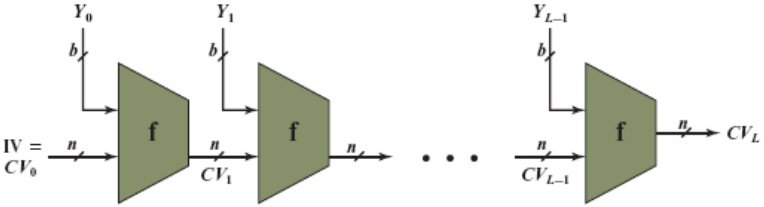
Requirements and Security

- Preimage
  - x is the preimage of h for a hash value $h = H(x)$
  - Is a data block whose hash function, using the function H, is h
  - Because H is a many-to-one mapping, for any given hash value h, there will in general be multiple pre-images
- Collision
  - Occurs if we have $x \neq y$ and $H(x) = H(y)$
  - Because we are using hash functions for data integrity, collisions are clearly undesirable

Example of a (weak) hash function

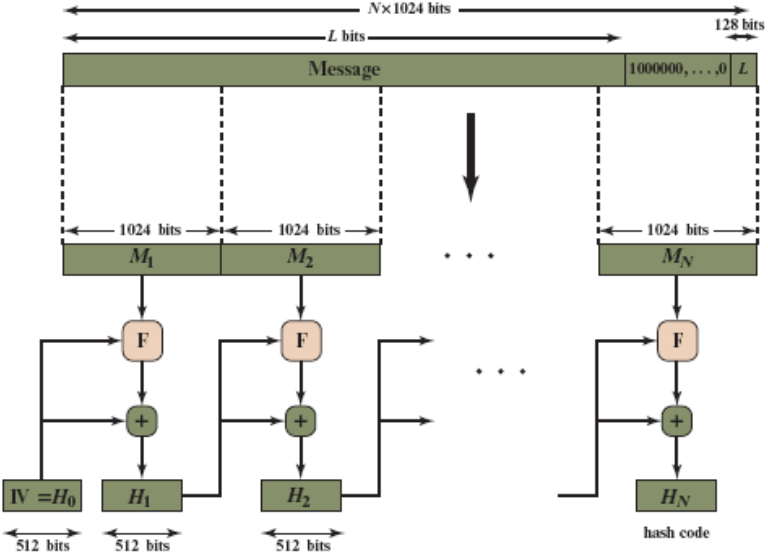
- Given a large prime number p and a primitive root g in $\mathbb{Z}_p^*$, with g the generator of $\mathbb{Z}_p^* = \{1 \ldots \text{p-1}\}$
- Let $H(x) = g^x \mod p$. Then H is a one-way function
- Example : $p = 7$, $g = 3$, $x \in 0 \ldots n$, then $H(x) = 3^x \mod 7 \in \{1 \ldots 6\}$
  - $H(0) = 3^0 \mod 7 = 1$
  - $H(3) = 3^3 \mod 7 = 64 \mod 7 = 1$
  - $H(9) = 3^9 \mod 7 = 262144 \mod 7 = 1$

General Structure of Secure Hash Code



IV = Initial value
CV_i = Chaining variable
Y_i = ith input block
f = Compression algorithm
L = Number of input blocks
n = Length of hash code
b = Length of input block

Message Digest Generation Using SHA-512



Applications of hashing functions

Randomness generation

- The output of a cryptographic hash function is assumed to be uniformly distributed
- Although this property has not been proven in a mathematical sense for common cryptographic hash functions, such as MD5, SHA-1, it is often used in the context of pseudo-random number generation (PRNG)
- Some NIST standards recommend to generate randomness based on hashing functions
- General idea: start with random seed, then hash iteratively
  - $b_0 = \text{seed}$
  - $b_{i+1} = H(b_i|\text{seed})$

Randomness generation

Encryption of data streams

- WiFi, xG require lighweight encryption
  - reduce electronics load and accomodate packet losses
- Generate a key stream as follows (OFB mode)
  - $k_0 = H(k_{A,B}|IV)$
  - $k_{i+1} = H(k_{A,B}|k_i)$
  - where $k_{A,B}$ is typically static (e.g. derived from a shared password) and IV is the initialization vector (nonce)
  - allows to compute any n-th key k_n
- Encrypt / decrypt
  - $Enc(m_i) = m_i \oplus k_i$
  - $Dec(c_i) = c_i \oplus k_i$

Authentication with a challenge-response

- Given only Alice and Bob know the shared secret k_{AB}
  - Alice: sends nonce n_A to Bob
  - Bob computes $r_{Bob} = H(k_{AB}, n_A)$, sends back to Alice
  - Alice computes $r_{Alice} = H(k_{AB}, n_A)$ and verifies $r_{Alice} \stackrel{?}{=} r_{Bob}$
- Mutual authentication can be achieved by a 2nd exchange in opposite direction
- Applications
  - HTTP digest authentication, Windows-based systems authentication
  - $k_{AB} = H(\text{password})$ and encrypt a challenge with k_{AB}
- This kind of authentication does not include negotiation of a session key
  - see in next chapters (key distribution)
- Alternatives ?
  - Public-key approach where Bob signs the nonce with his private key

Cooldown mechanism

- Similar to the *proof of work* approach
  - Alice selects a function $H()$ and a challenge (e.g., have five zeroes in the response).
  - Alice sends her selection and a value x to Bob.
  - Bob must find y such that $r = H(x|y)$ matches the challenge
  - When done, Bob sends y to Alice. Alice verifies that Bob solved the challenge
  - Bob is now allowed to perform an action at Alice's
- Predictable average complexity for a given $H()$ function
  - Birthday paradox
  - Alice could increase the complexity over time
- Force the client to put some effort in the protocol negotiation
  - useful against password guessing attacks, rate limitation of database updates, etc...
  - stateful but lightweight

Can hashing ensure integrity ?

- Hash function seen as a **fingerprinting** operation
- Take m, send $[m \mid H(m)]$
- .. but we may have a MiTM. The attacker
  - recovers m, modifies to m'
  - computes $H(m')$ (hashing functions is often negotiated in the handshake phase an known from everyone)
  - sends $[m' \mid H(m')]$
- Applying a hash function is not sufficient to secure a message.
  - $H(m)$ needs to be protected.

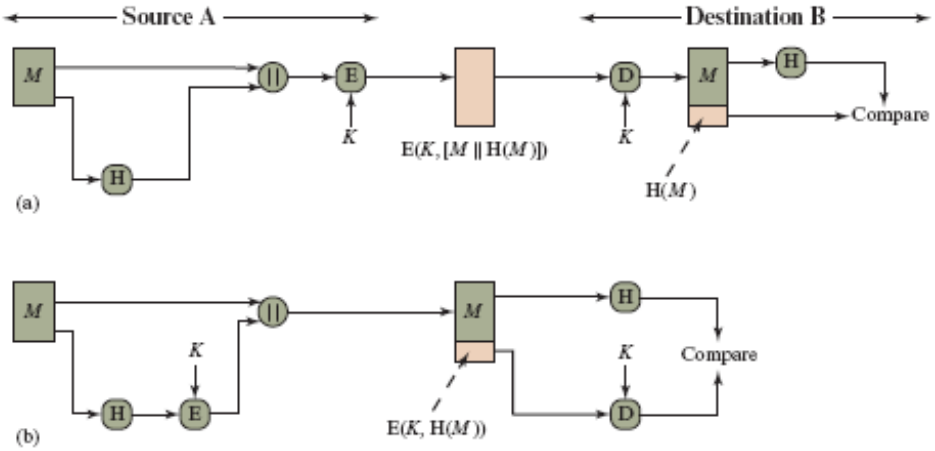
Message Authentication Code (MAC)

- Also known as a **keyed hash function**
- Typically used between two parties that share a secret key to authenticate information exchanged between those parties
- Idea
  - Alice, Bob agree on a common integrity key k
  - Alice generates m, sends $[m \mid H(m|k)]$
  - or a variant thereof (HMAC)...

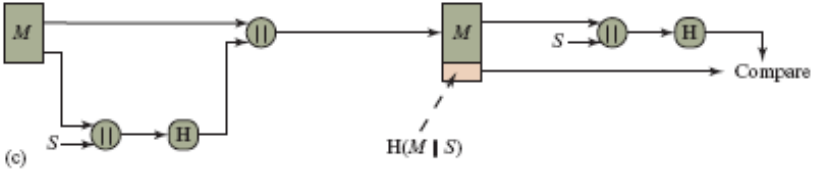
Message Authentication Code (MAC)

- Properties
  - Do prove message integrity.
  - Do detect tampering.
  - Cannot be forged.
  - Can be replayed.
- Do MACs prove authenticity?
  - It depends on the definition and scenario...
  - If k is unique between Alice and Bob. Similar to signcryption or authenticated MAC
  - If k is a shared group key used by Alice, Bob, Charlie, ... a receiver does not know who sent the message
  - An external observer (e.g., application-layer firewall) cannot validate the MAC as k is unknown to it

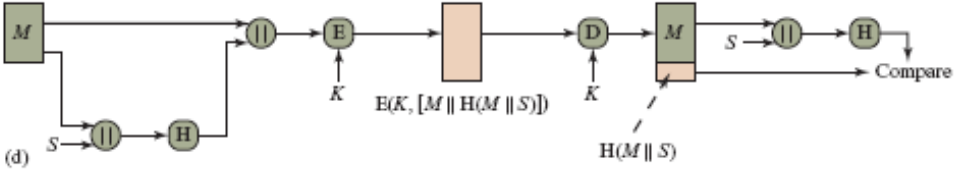
Examples of the Use of a Hash Function for Message Authentication



Examples of the Use of a Hash Function for Message Authentication



(c)



(d)

Discussion

- Is $sMAC(m) = E_k(H(m))$ a good MAC construction?
  - Let us assume we use a stream cipher with key s : $E_k(m) = m \oplus s$
  - Problem: attacker might know[1] m, so it can compute $H(m)$
  - The attacker now computes

$$sMAC(m) \oplus H(m) = E_K(H(m)) \oplus H(m) = H(m) \oplus s \oplus H(m) = s$$

  - Now the attacker has the key s. Furthermore can forge m' and

$$H(m') \oplus s = E_K(H(m')) = sMAC(m')$$

- So. . . . with a stream cipher certainly not (used in WiFi, 2G, . . .)

[1] handshake message, telephony signalling, standardized beacon, ...

Discussion

- Reasons for constructing MACs from cryptographic hash functions
  - Cryptographic hash functions generally execute faster than symmetric block ciphers
  - There are no export restrictions to cryptographic hash functions
  - Good hardware implementations (co-processor or SoC)
- Cryptographic concerns
  - The construction $H(K|m)$ and $H(m|K)$ are **not** secure
- The construction $H(K|m|K)$, called prefix-suffix mode, has been used for a while.
  - see for example RFC 1828
  - earlier implementations of the Secure Socket Layer (SSL) protocol (until SSL 3.0)
  - considered **vulnerable** to attack by the cryptographic community
- The construction $H(K|p|m|K)$ with p denoting an additional padding field does **not** offer sufficient security
- $HMAC_k(m) = h\Big((k \oplus opad) \mid\mid h\Big((k \oplus ipad) \mid\mid m\Big)\Big)$ **is** secure

Hash MACs (HMAC)

HMAC is the most used, lightweight, and secure construction

$$H(K \oplus opad | H(K \oplus ipad | m))$$

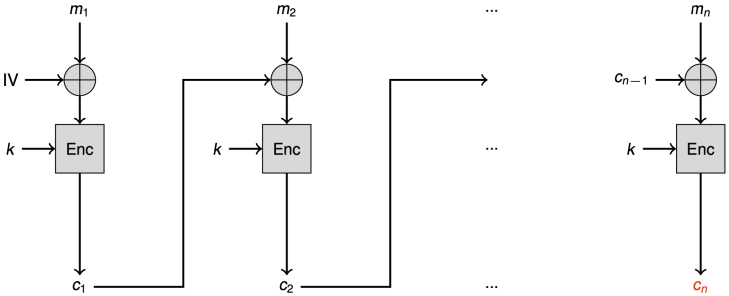
- The length of the key K is first extended to the block length required for the input of the hash function H by appending zero bytes.
- opad and ipad are two constants (crypto resistant) extended to match the message length
  - ipad $= 0\text{x}\mathbf{36}3636\ldots3636$
  - opad $= 0\text{x}\mathbf{5c}5c5c\ldots5c5c$
- The hash function is applied twice in a nested way.
- Currently no attacks have been discovered on this MAC function.

CBC-MAC

A CBC-MAC is computed by encrypting a message in CBC Mode and taking the last ciphertext block or a part of it as the MAC

m_1 m_2 ... m_n

CBC-MAC



- This MAC needs not to be mixed with a secret any further, as it has already been produced using a shared secret k.
  - This scheme works with any block cipher (AES, Twofish, 3DES, ...)
  - It is used, e.g., for IEEE 802.11 (WLAN) WPA2, many modes in SSL / IPSec use some CBC-MAC construction.

CBC-MAC

- CBC-MAC security
  - CBC-MAC must **not** be used with the same key as for the encryption
  - In particular, if CBC mode is used for encryption, and CBC-MAC for authenticity with the same key, the MAC will be equal to the last cipher text block
  - If the length of a message is unknown or no other protection exists, CBC-MAC can be prone to length extension attacks. CMAC resolves the issue.
- CBC-MAC performance
  - Older symmetric block ciphers (such as DES) require more computing effort than dedicated cryptographic hash functions, e.g. MD5, SHA-1 therefore, these schemes are considered to be slower.
  - However, newer symmetric block ciphers (AES) is faster than conventional cryptographic hash functions.
  - Therefore, AES-CBC-MAC is becoming popular.

References

- B. Coskun, N. Memon, Confusion/Diffusion Capabilities of Some Robust Hash Functions, CISS 2006: Conference on Information Sciences and Systems
- H. Krawczyk, M. Bellare, R. Canetti, HMAC: Keyed-Hashing for Message Authentication, Internet RFC 2104, February 1997.
- R. Merkle, One Way Hash Functions and DES, Proceedings of Crypto '89, Springer, 1989.
- Niels Ferguson, Bruce Schneier, Practical Cryptography, John Wiley & Sons, 2003
- Peter Selinger, http://www.mscs.dal.ca/ selinger/md5collision/
- P. Metzger, IP Authentication using Keyed MD5, IETF RFC 1828, August 1995
- R. L. Rivest. The MD5 Message Digest Algorithm, Internet RFC 1321, April 1992.
- M. Robshaw. On Recent Results for MD2, MD4 and MD5, RSA Laboratories' Bulletin, No. 4, 1996.
- Xiaoyun Wang, Yiqun Lisa Yin, and Hongbo Yu, Collision Search Attacks on SHA1, February 2005
- G. Yuval. How to Swindle Rabin, Cryptologia, July 1979.